Report of mini project step 1 of mandelbrot.

Mandelbrot set problem:

The mandelbrot set is a 2 dimensional set in the complex plane. The mandlebrot set is an uncountable set of values of c in the complex plane.
The mandelbrot can be partially calculated with a fixed resolution in a computational way. The algorithm is as follows:

Define fixed region dimensions (x,y)
Generate a grid of complex numbers over the fixed defined region with evenly spread x and y values.
For each point generate value: x+iy as c
For each point now:
    Initialize z = 0
    For n to max iterations (max iteration should be a fixed number)
        Compute: zn+1 = zn 2 + c
        If absolute value of zn+1 > 2: store zn+1 in given point and move to next point

        Else:
        If loop terminates before zn+1 > 2. Store max iterations' value

When loop is finished you have obtained the mandelbrot set in the form of a matrix with iteration counts

The problem is an embarrassingly parallel problem as it is highly paralizable since each value of each grid location can be calculated individually and does not rely on any other data than the complex grid generated equally for all locations.

The problem has many options for implementations. Some will perform better than others in terms of time to execute per loop.

Implementations:

For this first step of the mini project, these given implementations of computing the mandelbrot set has been introduced and performed:

1. Naive for loop
2. Vectorized using numpy
3. Njit using Numba

The exact implementation is included in the code in the form of a jupyter notebook, as well as their performance benchmarks.

1. Naive for loop:

The naive implementation features a nested for loop where each point is calculated sequentially. The outer loop goes from x = 0 -> x max width and the inner goes from y=0 -> y max height.

Each point is then calculated using the algorithm. If zn+1 > 2, that point zn+1 is saved, otherwise max iter is saved.

When the outer loop terminates, the function terminates and returns the mandelbrot matrix.

2. Vectorized using numpy

The vectorized solution using numpy uses numpy arrays which forces the array to be located close by in memory to reduce cache misses.

Additionally the numpy vectorized solution calculates the next z value for all grid items each time and keeps track of which locations in the grid terminate from the loop via using a mask matrix.

The mask matrix is a matrix of the same dimensions as the mandelbrot matrix, containing boolean values. When a point in a location escapes with having a zn+1>2. The mask updates that point location to signal that the given cell is completed and does not need any further calculation applied in the next iteration.

When max iteration is achieved the function terminates and return the mandelbrot matrix

3. Njit using Numba

The Njit using Numba solution uses the naive implementation but applies a handler (@njit) from the Numba library to perform a Just In Time compilation on the function. This means that the function will be precompiled for later usage and will not run interpretively as the naive version.

Numba does not support all of NumPy, as NumPy uses high-level Python abstractions and dynamic features that rely on the CPython interpreter, which cant reliably be compiled to efficient machine code by Numba.

Benchmarking methodology:

The benchmarking utilized %timeit in jupyter notebook environment. Comparison of all %timeit measurements is stored in a csv file in the submitted folder.

To measure the performance when scaling the problem size. The given resolutions of the grid has been tested with each implemented solution:

1024x1024
2048x2048
4096x4096
8192x8192

Each test ran on a local laptop machine. No cloud machine was used for the benchmark.

PC specs:

=== Static System Information ===

System: Linux 6.17.0-14-generic (#14~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Jan 15 15:52:10 UTC 2)
Machine: x86_64
Processor: x86_64
CPU Brand: Intel(R) Core(TM) i7-7820HQ CPU @ 2.90GHz
CPU Arch: X86_64
CPU Bits: 64
Number of Physical Cores: 4
Number of Logical Cores: 8

=== CPU Cache Sizes ===
L1 Data Cache: 131072
L1 Instruction Cache: 131072
L2 Cache: 1048576
L3 Cache: 8388608

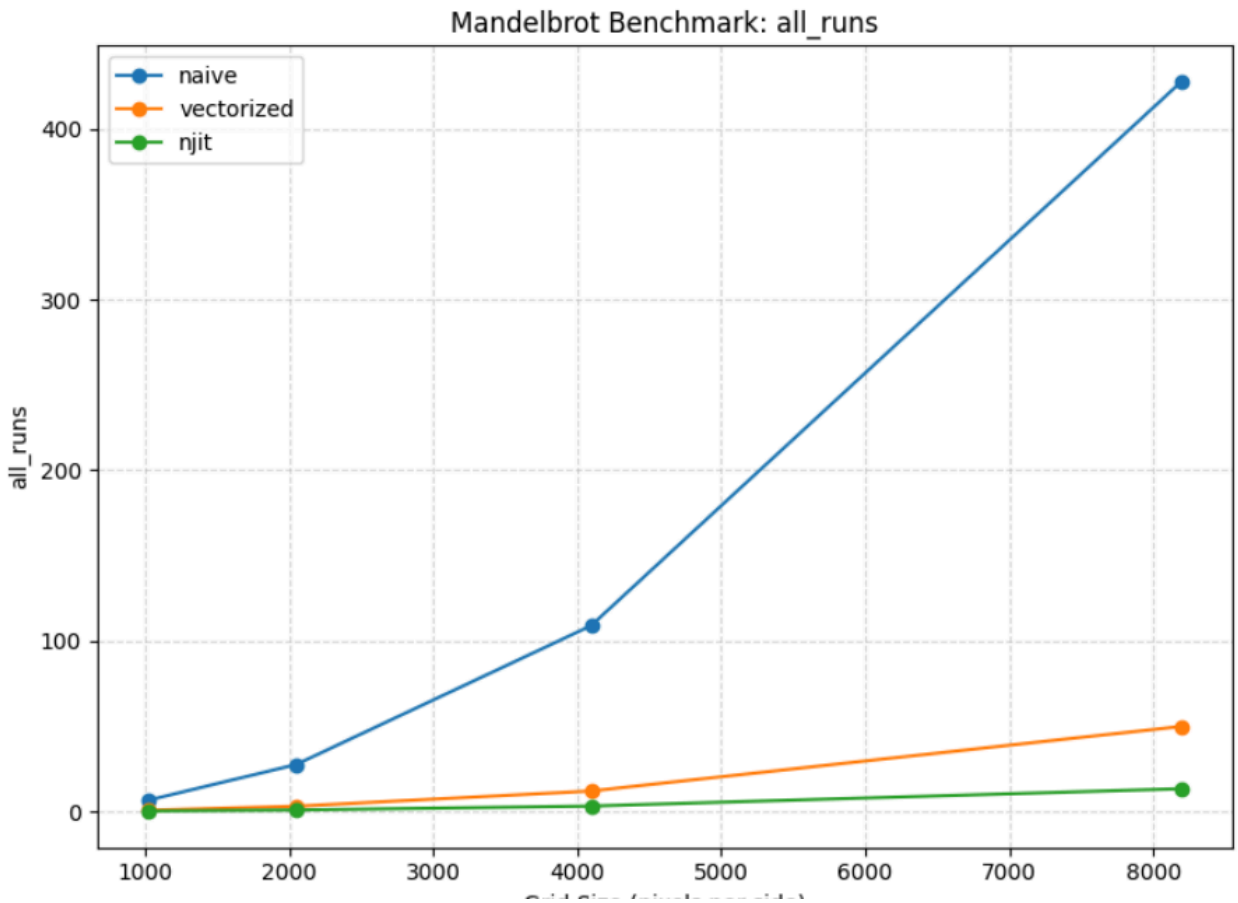
=== Memory ===
Total RAM: 31.20 GB

Benchmarking results

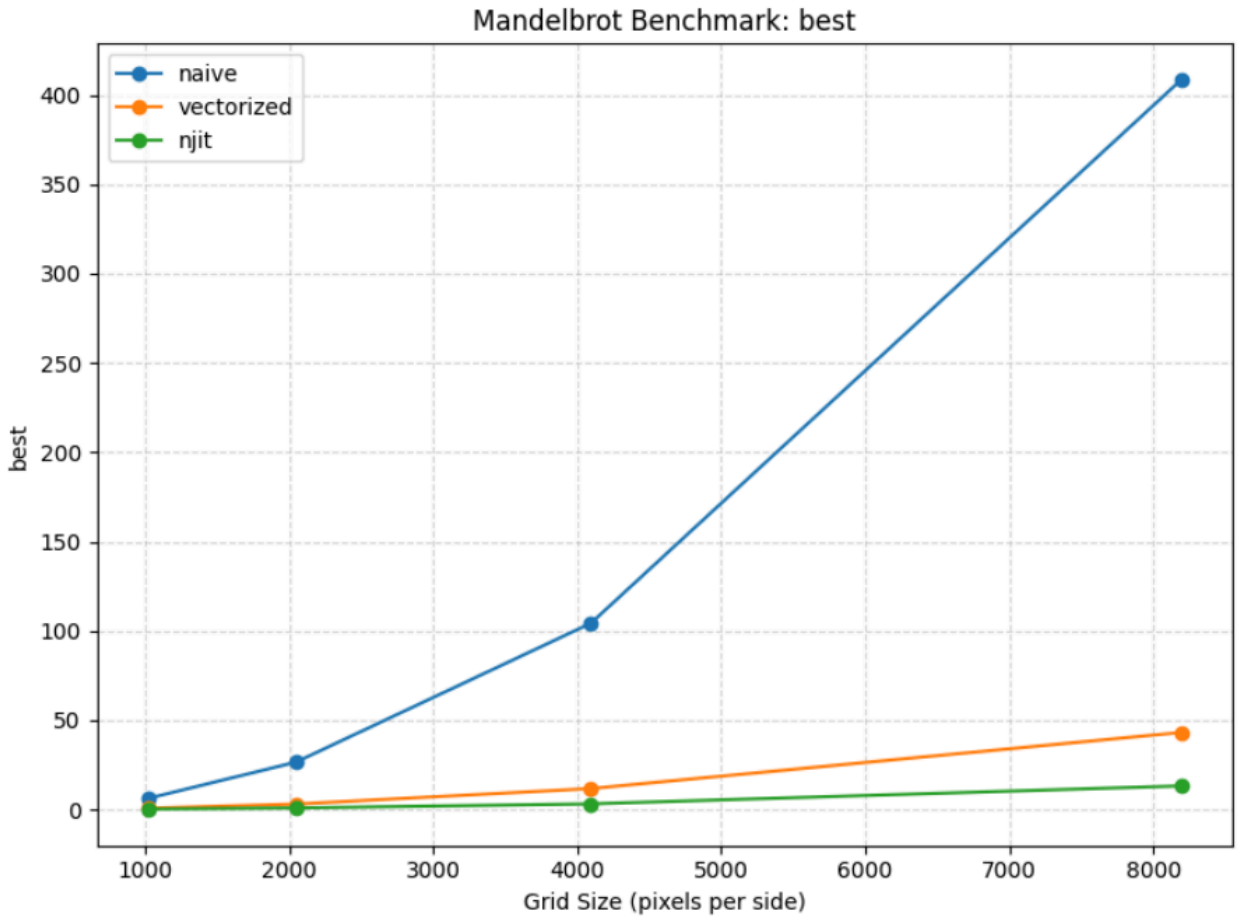
4 main benchmarks were for each implementation in the experiments:

1. Average execution time per loop.
2. Best execution time per loop.
3. Worst execution time per loop.
4. Compilation time.

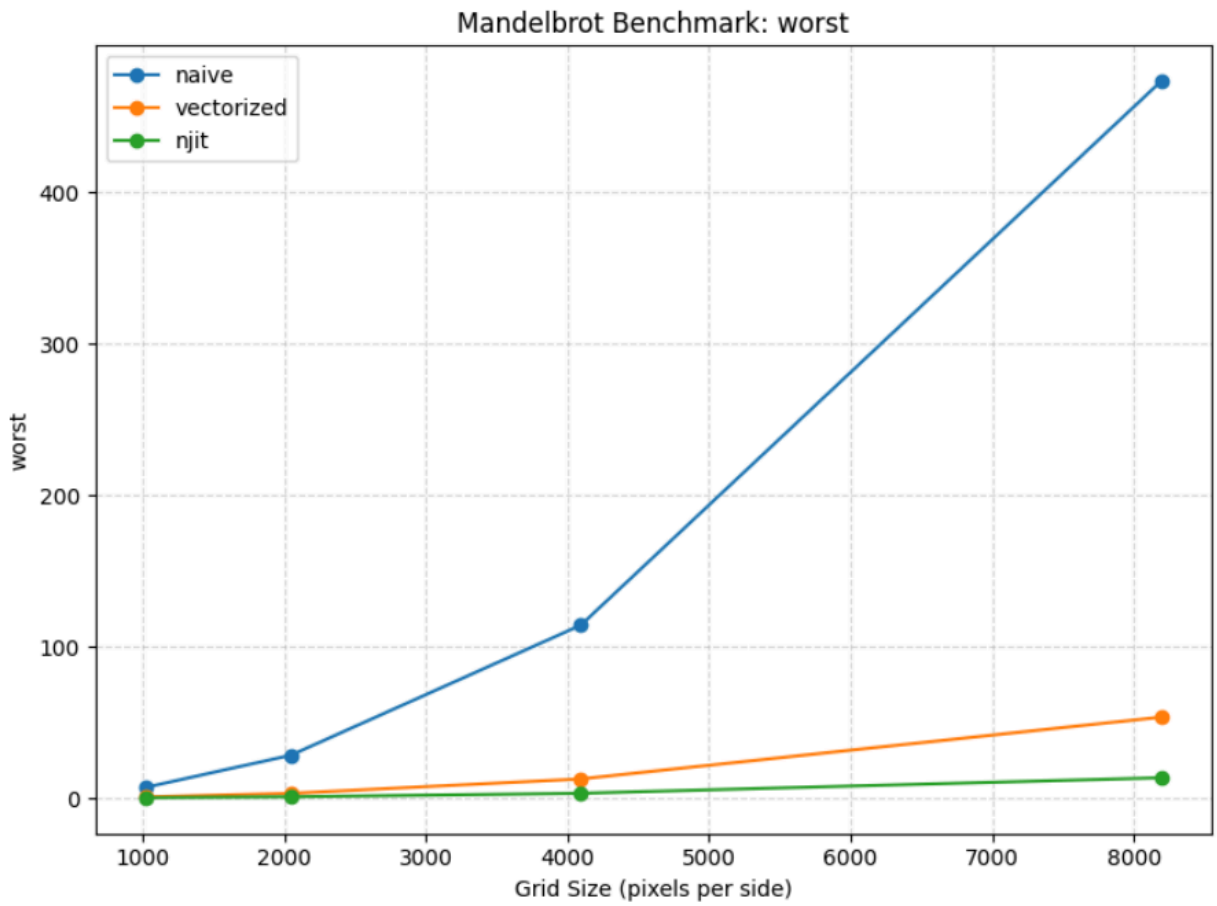
Average time for each loop.
Y axis is the average time per loop.
X axis is the problem size (grid size).



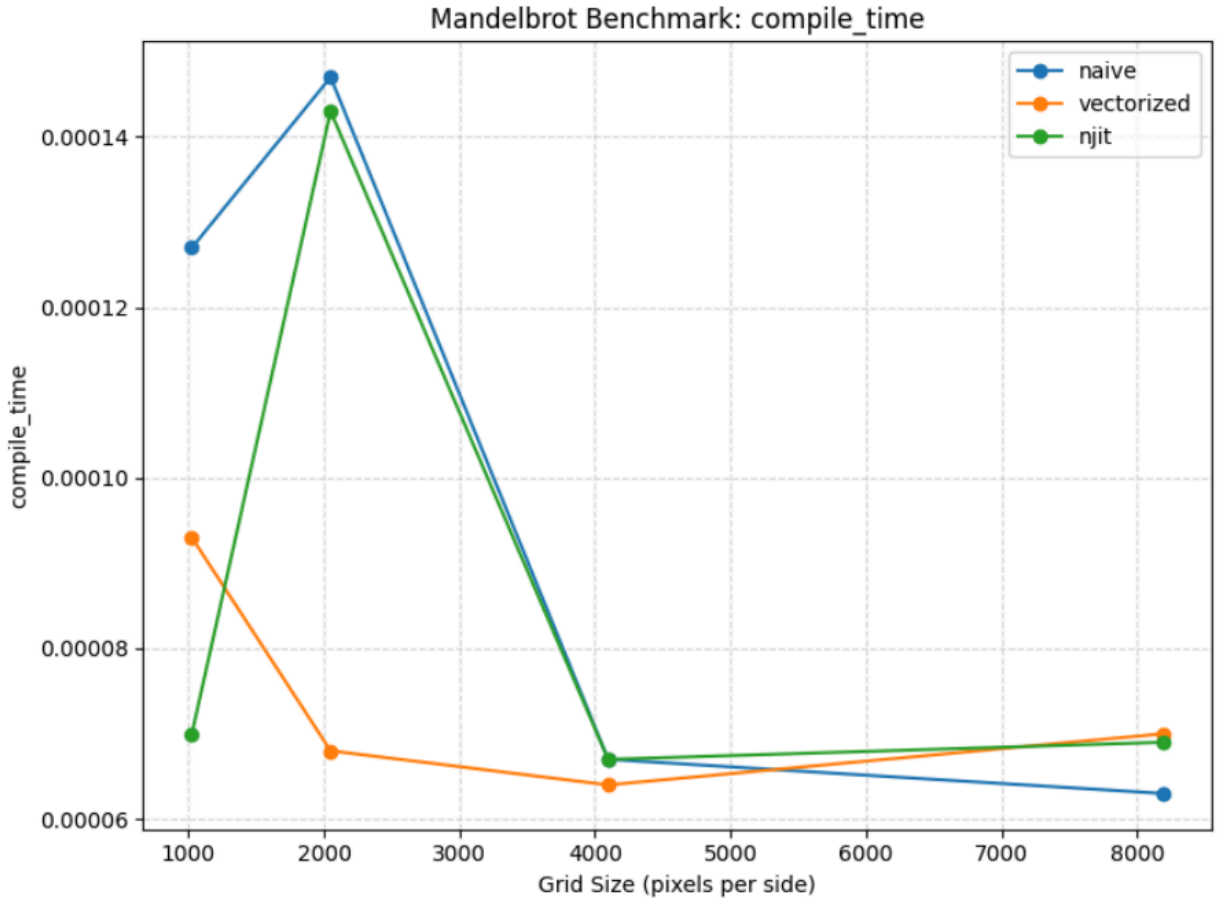
Best time for each loop.
Y axis is the best time per loop.
X axis is the problem size (grid size).



Worst time for each loop.
Y axis is the worst time per loop.
X axis is the problem size (grid size).



Compile time for each execution.
Y axis is the compilation time.
X axis is the problem size (grid size).



For specific numbers of time measurements during the experimentation runs see attached jupyter notebook file (pdf or ipynb) or see the benchmark.csv file.

Evaluation of benchmarking results.

The benchmark results show a clear pattern that the naive solution performs much worse than the vectorized version and the njit (Just In Time compilation) version.

This result was expected.

As the problem size grows so does the time per loop for each implementation. However as the problem size grows linearly, the execution time grows exponentially. In the worst case the naive version grows from:
6.64 s ± 300 ms per loop -> 7min 7s ± 21.3 s per loop

The vectorized version performs way better than the naive version, due to its smarter use of numpy arrays eliminating a lot of cache misses. It performs on average 8-9 times faster than the naive version.

However the best solution was the njit version, utilizing pre compiled functions to accelerate repeated calls to the computer hardware for optimized performance of operations.

The njit version performs on average 30 times faster than the naive version and around 3 times faster than the vectorized version.